SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I Mukunddhan VR *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MUKUNDHAN

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ARTIFICIAL INTELLIGENCE

CO2 – Scenario Based Assignment | Assessment Tool 1

Course Outcome CO2: Apply search, optimization, constraint satisfaction, and game-solving techniques to complex AI problems.

1. AI-Powered Emergency Medicine Drones

i. Greedy Best-First Search: behaviour, strengths and weaknesses

Greedy Best-First Search selects the location that appears closest to the destination according to the heuristic, here straight-line distance. It can find a route quickly and is useful when time is critical. However, it ignores the actual cost already travelled, so it may choose a short-looking route with dangerous weather, difficult terrain, or blocked paths. It is not guaranteed to find the least-cost route.

ii. How A balances cost and heuristic*

A* evaluates each node using **f(n) = g(n) + h(n)**, where g(n) is the cost from the start and h(n) estimates the remaining cost. This balances the cost already incurred with the estimated distance to the medicine destination. If movement costs are represented accurately and the heuristic is admissible, A* can find an optimal route in a stable search problem.

iii. Impact of heuristic accuracy

A realistic heuristic helps both algorithms focus on promising routes. If the heuristic is inaccurate, Greedy Search may be attracted to a route that looks close but is expensive or unsafe. A* is also affected because its estimated remaining cost influences node ordering, although its g(n) term accounts for costs already travelled. An admissible heuristic that does not overestimate the remaining cost supports A* optimality under standard conditions.

iv. Effect of blocked paths and dynamic changes

When a path becomes blocked, the current route may no longer be valid. The drone must receive updates, remove or penalize blocked edges, and replan. Repeatedly running a search from scratch may take time; an incremental replanning method can reuse earlier search information. Stale or incomplete updates can still lead to delay and risk.

v. Recommended algorithm

A* is the better basic choice because it considers both route cost and estimated remaining distance, supporting low-cost routing when its assumptions hold. For a real-time drone in a changing flood, it should be combined with frequent sensor updates, safety constraints, and fast replanning (such as an incremental A* variant). The drone should reject unsafe routes even if they are short. This balances efficiency, route quality, and safety.

2. Smart-City Traffic Signal Optimization

Formulation as an optimization problem

Represent signal timings and phases at intersections as decision variables. Minimize a combined objective such as total vehicle waiting time, fuel consumption, and congestion, subject to constraints on signal-cycle lengths, minimum green times, pedestrian crossings, and safe phase order. The objective can be written as **J = w■W + w■F + w■C**, where W, F, and C represent waiting, fuel use, and congestion, and the weights express their relative importance. Traffic data changes continuously, so the solution must be updated.

i. Hill Climbing and its limitations

Hill Climbing starts with a signal plan and repeatedly changes it to a neighbouring plan if the change improves the objective. It is simple and can improve a plan quickly. However, it may stop at a local optimum, depends on the starting plan, and may not explore enough alternatives in a large search space. Frequent traffic changes can also make its current solution outdated.

ii. Local maxima, plateaus and ridges

A **local maximum** (or local optimum for a minimization problem) is a plan better than nearby alternatives but not the best overall; for example, improving one busy junction while worsening nearby roads. A **plateau** occurs when many neighbouring plans have similar scores, so the algorithm has no clear direction. A **ridge** is a narrow path of improvement requiring coordinated changes across several intersections; single small changes may fail to follow it.

iii. How Simulated Annealing or Genetic Algorithms help

Simulated Annealing sometimes accepts a worse solution early in the search, allowing it to escape local optima; this acceptance becomes less likely as the temperature decreases. A Genetic Algorithm maintains a population of signal plans and uses selection, crossover, and mutation to explore different combinations. Both can search more broadly than basic Hill Climbing, though they require suitable parameter settings and computation time.

iv. Recommended approach

A Genetic Algorithm is suitable for exploring coordinated timings across many intersections because a population can represent diverse city-wide plans. Use a fitness function based on weighted waiting time, fuel consumption, and congestion, while enforcing safety and signal constraints. To meet real-time needs, optimize periodically or in rolling time windows, use current traffic data, and apply a fast local adjustment method between broader optimization runs. This offers scalability and adaptability, although performance must be tested on realistic traffic data.

3. Mars Rover in Unknown Terrain

i. Why an online search agent is required

An offline search agent assumes it has a sufficiently complete model before planning. The Mars rover has limited sensing and does not know the full terrain in advance. An online search agent can act, observe the result, update its knowledge, and choose its next action. This is appropriate for exploration where information becomes available during movement.

ii. Partial observability and dynamic environments

The rover may not detect a crater or loose rock until it is close, and its sensors may provide incomplete or uncertain readings. Terrain, lighting, or communication conditions can also change. These factors make a precomputed path unreliable. The rover must account for uncertainty, maintain a safety margin, and replan when new observations conflict with its current map.

iii. Building and updating the internal model

The rover combines sensor readings, camera images, odometry, and previous observations to build a local map of safe and hazardous areas. After each movement, it compares the expected result with what actually happened and updates its position estimate and terrain model. Newly detected hazards are marked as obstacles, and newly observed safe areas become possible routes. The model remains incomplete and is refined as exploration continues.

iv. Online search compared with uninformed and informed search

Uninformed search, such as breadth-first search, uses no heuristic guidance and may explore many unnecessary states. Informed search, such as A*, uses a heuristic to prioritize promising routes, but generally benefits from a known map and cost model. Online search focuses on acting and learning when the environment is not fully known; it can use heuristic guidance while updating the model during exploration.

v. Most suitable approach

A safety-focused online agent using heuristic-guided local planning and continual replanning is most suitable. It should prefer routes with low estimated travel cost while avoiding uncertain or hazardous terrain, and stop or choose another route when sensor confidence is low. This approach adapts to new information and supports safe exploration, although limited sensing and processing mean that it cannot guarantee a globally optimal route in every situation.

4. University Exam Timetabling as a CSP

Formulation: variables, domains and constraints

A Constraint Satisfaction Problem (CSP) consists of variables, domains, and constraints. **Variables:** the exam assigned to each course. **Domains:** available time slots and suitable halls for each exam. **Constraints:** students cannot have overlapping exams; hall capacity and availability must be respected; required subject ordering must be maintained; faculty must be available; and approved accommodations must be provided. A valid timetable is an assignment that satisfies all hard constraints.

ii. Backtracking search

Backtracking assigns a time and hall to one exam at a time. After each assignment, it checks whether the constraints remain satisfied. If a conflict occurs, it reverses the latest assignment and tries another option. Choosing the most constrained exam first and trying the least-constraining option can reduce unnecessary backtracking.

iii. Constraint propagation and forward checking

Forward checking removes domain values that become impossible after an assignment. For example, when an exam is placed in a slot, that slot can be removed from the domains of exams that share students or require the same unavailable hall. If a variable's domain becomes empty, the algorithm detects the conflict early and backtracks rather than continuing with an impossible partial timetable.

iv. Real-world challenges and scalable strategy

Large universities may have thousands of students, many courses, room capacities, faculty schedules, and special accommodation requirements. Constraints can interact, making the search space large. A scalable strategy is to combine constraint propagation with backtracking, the minimum-remaining-values heuristic, and degree-based variable ordering. For very large cases, decomposition or local improvement can help, while hard constraints must always be preserved and the final timetable checked for conflicts.

5. Game AI: Minimax and Alpha-Beta Pruning

i. How Minimax models the game

Minimax represents the game as a tree of possible states and moves. The AI is the maximizing player, trying to obtain the highest score, while the opponent is assumed to minimize that score. Terminal states receive utility values, and these values are propagated upward: maximizing nodes select the largest child value and minimizing nodes select the smallest. The AI chooses the move with the best backed-up value.

ii. Computational challenges of large game trees

If the average number of legal moves is b and the search depth is d, ordinary Minimax may examine about O(b■) nodes. The number grows rapidly as depth or branching factor increases, causing delays and high memory or processing demands. In a real-time strategy game, the AI may need to act before it can search to terminal states.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning gives the same decision as Minimax for the searched tree but skips branches that cannot affect the final choice. Alpha is the best value found so far for the maximizing player; beta is the best value for the minimizing player. When alpha is greater than or equal to beta, further exploration of that branch is unnecessary. For example, if the minimizing player can already force a value no better than a choice the maximizing player has, that branch need not be explored. Good move ordering increases pruning; worst-case complexity remains O(b■), while ideal ordering can reduce it toward O(b■$^{/2}$).

iv. Evaluation function

A practical evaluation function could be **Eval(s) = w■M + w■R + w■T + w■O** – **w■D**, where M is the difference in military strength, R is resource advantage, T is territory or strategic-position advantage, O is objective progress, and D is damage or vulnerability. The weights reflect the importance of each factor. The function should be tested so that its scores correlate with useful strategic outcomes, rather than favouring one factor at the expense of winning.

v. Strategy for optimality and real-time performance

Use depth-limited Minimax with Alpha-Beta pruning, good move ordering, and an evaluation function for non-terminal states. Iterative deepening searches depth 1, then 2, and so on, providing a usable best move if time runs out. A time limit and safe fallback move ensure responsiveness. This approach gives strong decisions within a deadline, though it does not guarantee a globally optimal move when the search is cut off before reaching terminal states.

Prepared to match the assignment's rubric: scenario understanding, key issues, concept application, justified decisions, and clear structure.